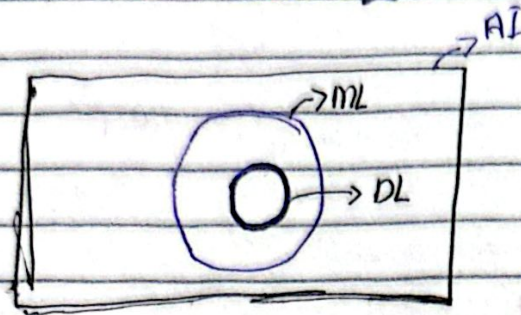


Deep Learning



- DL $\Rightarrow$ • Mimicing human brain
 - The way human learn things
 - Neural Networks (Multi Layered Neural Networks)

(1) ANN: Artificial Neural Network $\left\{ \begin{array}{l} \rightarrow \text{Classification} \\ \rightarrow \text{Regression} \end{array} \right.$

(2) CNN: Convolutional Neural Network: when input data is in form of images, video frames

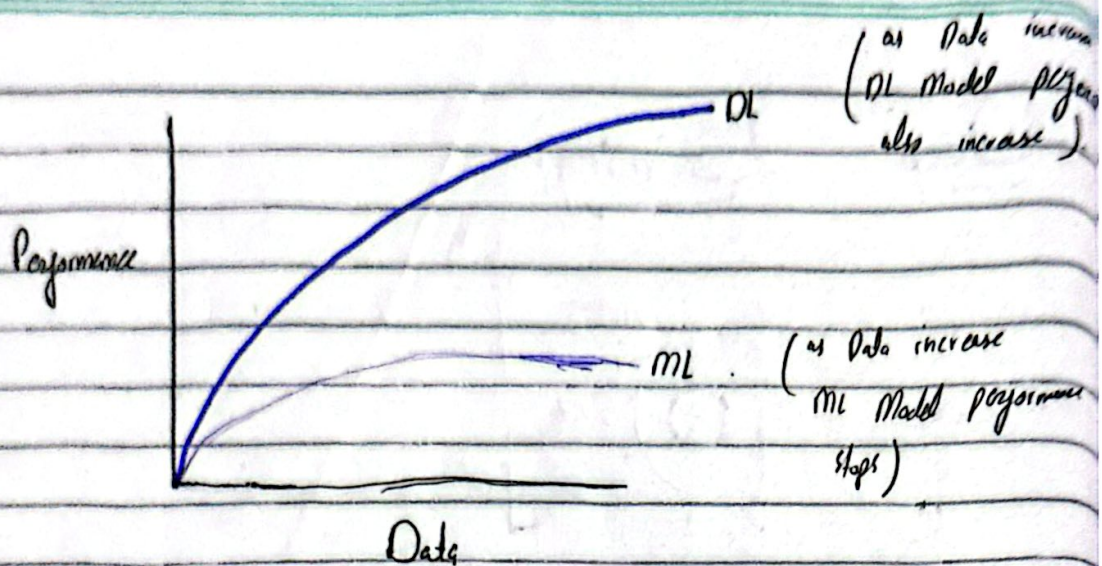
$\left. \begin{array}{l} \rightarrow \text{RCNN} \\ \rightarrow \text{Masked RCNN} \end{array} \right\} \begin{array}{l} \text{image segmentation} \\ \& \text{object detection} \\ (\text{YOLO}) \end{array}$

(3) RNN: Recurrent Neural Network: input is in Text, Time series (NLP)

$\rightarrow$ word embeddings

$\rightarrow$ LSTM RNN, GRU RNN, Bidirectional LSTM RNN

$\rightarrow$ Text classification



Perceptron

(Single Layered Neural Network)

* Artificial Neuron

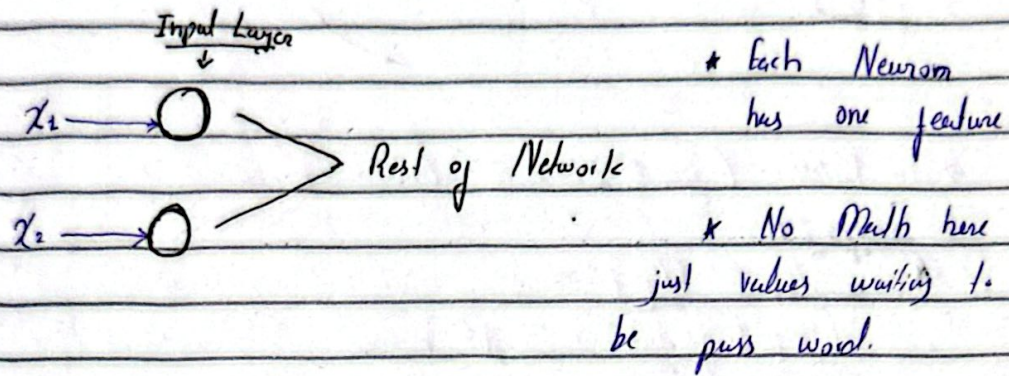
* Used to Solve Binary Classification.

Dataset

IQ	Input Features		Pass/Fail
	x_1	x_2	
→ 95		Study hr	
		3	0
→ 110		4	1
→ 100		5	1

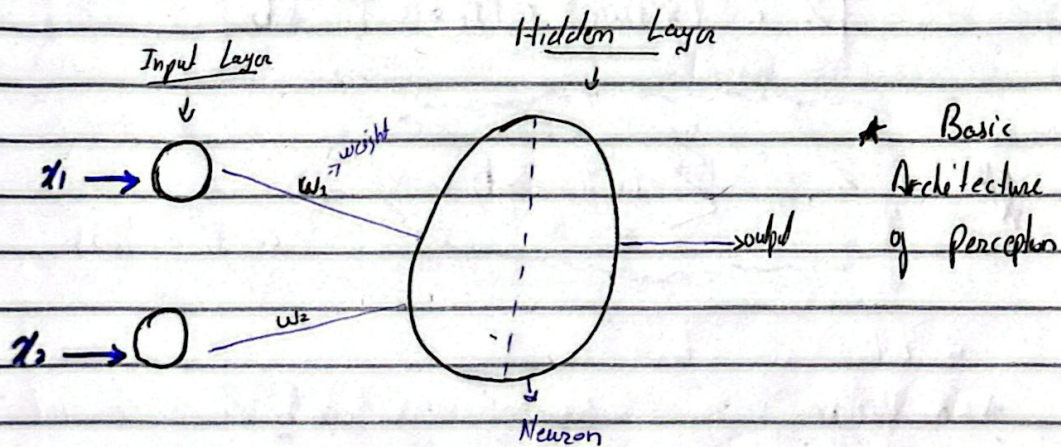
1: Input Layer:

Door Through which Data Enters a Network



* We don not count input layer (& its Neurons).

2: Hidden Layer : Brain (where logic & Maths rules)



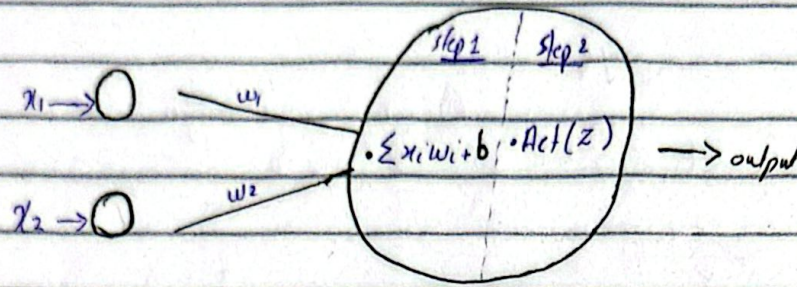
* We only count hidden layer

* In hidden Layer I only have One Neuron

* In this hidden layer we specifically perform 2 Tasks & then get the output

Input Layer

Hidden Layer



→ In hidden layer → 2 Task/steps Occurs.

Step 1:

★ In Step 1 → Summation occurs of
(input feature × Weight)

$$(x_1 \times w_1)$$

$$(x_2 \times w_2)$$

★ As we have only Two input features

$$Z = (x_1 \times w_1) + (x_2 \times w_2) + b$$

★★★

$$Z = \sum_{i=1}^n w_i x_i + b$$

∴ b = bias

⇒ what is bias?

★★★ Weights are initialized randomly, so what if weight equals zero, then so without "b" equation will become zero & no processing will happen

★★★ So this "bias" need to have some value i.e. $b = 1$, so that entire process doesn't become zero.

→ bias is treated as Noise (in Linear Regression terms it is the intercept).

Step 2: Activation Function:

* Maths Function applied to "z".

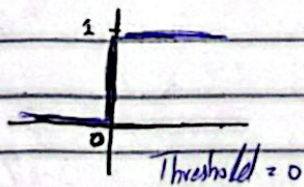
* Main Aim of Activation function is to Transform Output of z.

e.g. I wanna Transform the output b/w
→ 0 to 1
→ -1 to 1 etc

& We can use different Activation function for that like Step function, sigmoid function

→ Since we are solving binary classification so 0 to 1

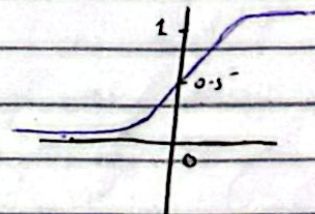
Step Function:



$$\begin{cases} 0 & \text{if } z \leq 0 \\ 1 & \text{if } z > 0 \end{cases}$$

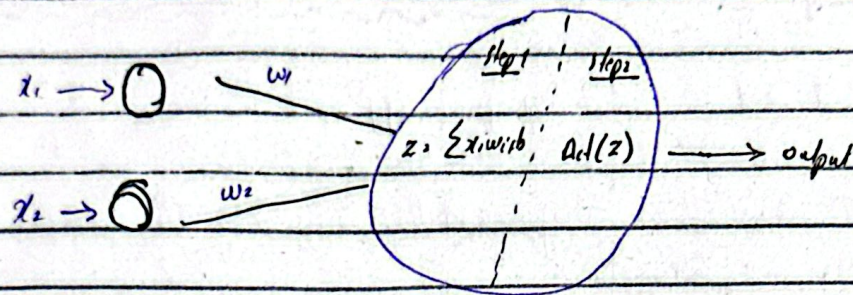
↑
output

Sigmoid Function



$$\begin{cases} 1 & \text{if } z > 0.5 \\ 0 & \text{if } z < 0.5 \end{cases}$$

→ In both cases we are able to change value b/w 0 to 1 just the threshold is different.



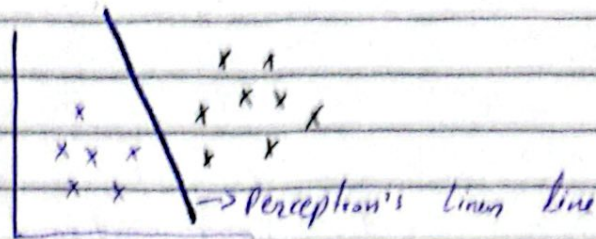
	T_d	Study hr	output
Row 1	95	3	0
Row 2	110	4	1
Row 3	100	5	1

So we pass row 1 through perceptron & get output "0" & real output is also "0" means No error → The weights are good

if we pass row 1 & get "1" & real output "0" means There is "error" & we need to update the weights

(This process occurs until we get optimal weights)

* Perceptron is a Linear classifier
it draws Linear line b/w categories.



also $z = \sum_{i=1}^n x_i w_i + b$

it's similar to linear regression equation

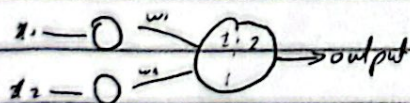
$$\left. \begin{aligned} y &= mx + c \\ z &= wx + b \end{aligned} \right\} \text{almost same used for linear line.}$$

x ————— x

Perceptron Models

Single Layered
Perceptron Model

* What we studied



→ This is called Feed Forward
Neural Network

→ Only good for linearly separable problems

Multi-Layered
perceptron Model

* ANN

→ Multiple hidden
layers

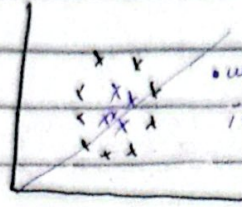
* Forward & Backward pass

* Loss function

* Activation function

* Optimizer

e.g. This problem



• we cannot separate it using linear line

• So we need to use Multi-Layered Neural Network

x ————— x

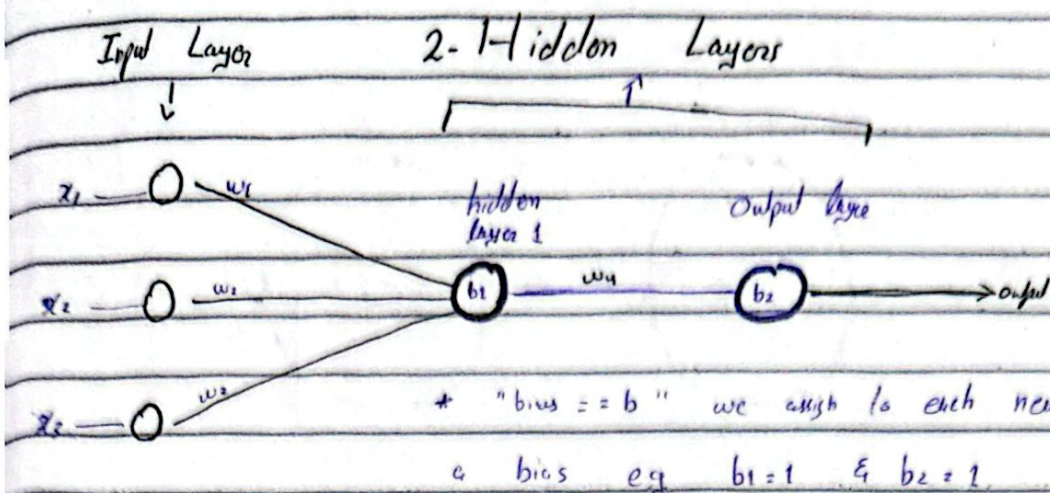
Multi-Layered Perceptron Model:

* Multi-Layered Neural Network

⇒ (Artificial Neural Network
ANN)

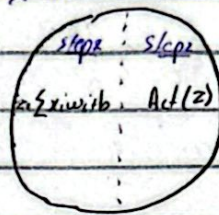
Dataset:

x_1	x_2	x_3	y
IQ	Study hrs	Play hrs	Output
95	4	4	1
100	5	2	1
95	2	7	0



* As I have Two Neurons so its a 2-Layered Neural Network

* In each hidden layer neuron The same two steps occurs →

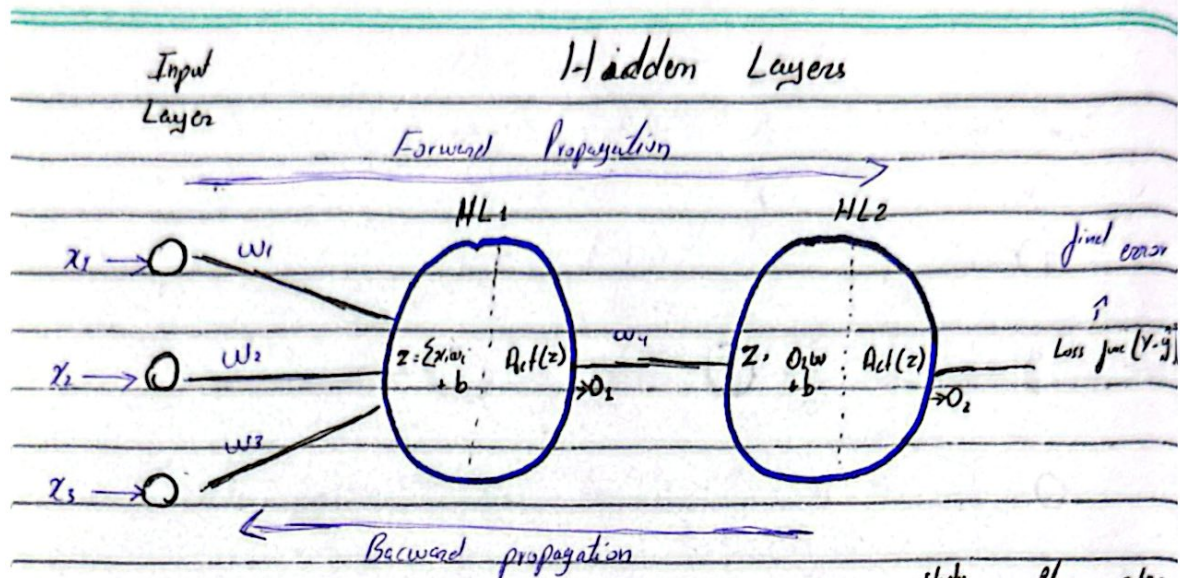


* Also each hidden layer neuron will give us output lets say output of 1st == O_1 & 2nd == O_2 .

* Output of hidden layer 2 (output layer) will be the output of this model.

* Final error ⇒ Actual output - predicted output
& try to minimize the error by updating the weights.

* The update process occurs from back to front i.e. "←". 1st update " w_4 " then " w_1 ", " w_2 " & " w_3 ". This is called Back Propagation.



	Id	study hr	play hr	sleep
row 1 $\Rightarrow$	95	4	4	1
row 2 $\Rightarrow$	100	5	2	1

(1) Forward propagation:

$\Rightarrow$ 1-Hidden Layer 1:

* Let's say we have randomly initialized weights:

$$\begin{matrix} w_1 & w_2 & w_3 \\ [0.01, & 0.02, & 0.03] \end{matrix}$$

* Let's say $b_1 = 0.001$ & $b_2 = 1$

* We pass row 1 of data set.

* In hidden Layer 1 There are Two Steps.

$\rightarrow$ Step 1:

$$z = \sum_{i=1}^n x_i w_i + b$$

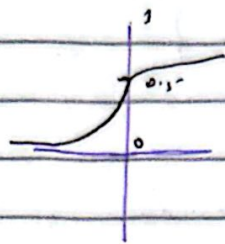
$$z = x_1 w_1 + x_2 w_2 + x_3 w_3 + b_1$$

$$z = 95 \times 0.01 + 4 \times 0.02 + 4 \times 0.03 + 1 \times 0.01$$

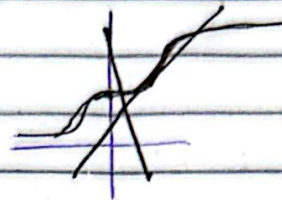
$$z = 1.151$$

→ step 2: Activation (z)

* we will use sigmoid function



$$f(z) = \frac{1}{1 + e^{-z}}$$



$$f(z) = \frac{1}{1 + e^{-1.51}} = \boxed{0.754}$$

So output of hidden Layer 1 "O₁" is $\boxed{0.754}$

* This O₁ will pass to hidden Layer 2.

→ Hidden Layer 2:

* We have randomly initialized w₄:

$$[0.02]$$

& bias b₂ is $[0.03]$

$$\text{So } w_4 = 0.02$$

$$b_2 = 0.03$$

→ Step 1:

$$z = O_1 \times w_4 + b_2$$

$$= 0.754 \times 0.02 + 0.03$$

$$\boxed{z = 0.04578}$$

→ Step 2:

$$\text{Activation (2)} = \frac{1}{1 + e^{-z}}$$

$$= \frac{1}{1 + e^{-10.04518}}$$

$$= 0.51129$$

* So output of Hidden Layer 2 " O_2 " is 0.51129

$$O_2 = 0.51129$$

*** O_2 is the predicted output of this particular model.

$$\text{So } O_2 = \hat{y} = 0.51129$$

*** As we had passed row 1 so see its real output. The real output is "1"
& I got "0.51129"

** Find error $\Rightarrow$ (True - predicted) or $(y - \hat{y})$
also called Loss Function:

$$\begin{aligned} \text{Loss function} &= (y - \hat{y}) \\ &= (1 - 0.51129) \end{aligned}$$

$$\text{Loss function} = 0.49$$

* We need to minimize this error.

* To minimize we need to update all the weights.

Backward Propagation: Update of weight occurs in reverse order i.e. from end to start. (Right to Left).

1st update w_4 then w_1, w_2, w_3

← Backward Propagation:

*** After updation "Next Input" will be set i.e. row 2 will be given & all the steps will repeat.

x ————— x

Loss Function Vs Cost Function.

⇒ $(y - \hat{y})^2$

* Calculate error for each & every point (input) & update the weight.

* For every input, we do the Forward & backward propagation

e.g. $MSE = (y - \hat{y})^2$

⇒ $\sum_{i=1}^n (y - \hat{y})^2$

* Calculate error at once (sum for all points) & then update the weight.

* We don't calculate for every point & update the weight.

* Forward for all then backward for these summation & upon
e.g. $MSE = \sum_{i=1}^n (y - \hat{y})^2$